UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623
Database Programming
2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	LAM YOKE YU
Matric Id	:	A23CS0233
Section	:	02

Name	:	TAN YI YA
Matric Id	:	A23CS0187
Section	:	02

Task 1: Database Creation and Integrity Constraints

1. Create database

```sql
CREATE DATABASE host_mgmt_yokeyu_yiya;
USE host_mgmt_yokeyu_yiya;
```

2. Create tables with entity and referential integrity

3. Apply constraints

```sql
CREATE TABLE room_types(
    type_id INT PRIMARY KEY,
    type_name VARCHAR(10) NOT NULL,
    rent DOUBLE NOT NULL,
    deposit DOUBLE NOT NULL,
    capacity INT NOT NULL
);

CREATE TABLE rooms(
    room_id INT PRIMARY KEY,
    room_no VARCHAR(5) NOT NULL,
    floor_no INT NOT NULL,
    is_occupied BOOL DEFAULT FALSE,
    type_id INT NOT NULL,
    FOREIGN KEY (type_id) REFERENCES room_types(type_id)
);

CREATE TABLE students(
    student_id INT PRIMARY KEY,
    fname VARCHAR(20) NOT NULL,
    lname VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
    checkin_date DATE,
    room_id INT,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);

CREATE TABLE maintenance(
    maint_id INT PRIMARY KEY,
    room_id INT NOT NULL,
    issue_desc TEXT NOT NULL,
    severity VARCHAR(10),
    status VARCHAR(10) NOT NULL,
    reported_on DATE NOT NULL,
    resolved_on DATE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

```sql
CREATE TABLE payments(
    payment_id INT PRIMARY KEY,
    student_id INT NOT NULL,
    amount DOUBLE NOT NULL,
    paid_on DATE NOT NULL,
    method VARCHAR(5) NOT NULL,
    note TEXT,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

5. Schema maintenance

5.1 Alter table

```sql
ALTER TABLE students
    ADD COLUMN email VARCHAR(30) UNIQUE;
```

5.2 Drop table

```sql
CREATE TABLE testing(
    test_id INT PRIMARY KEY,
    test_desc TEXT NOT NULL
);

DROP TABLE testing;
```

Task 2: Data Manipulation and Filtering

1. Data Insertion (DML):

```
INSERT INTO room_types (type_id, type_name, rent, deposit, capacity) VALUES
(1, 'Single', 550, 300, 1),
(2, 'Double', 700, 350, 2),
(3, 'Premium', 900, 450, 2),
(4, 'Family', 1200, 600, 4),
(5, 'Economy', 450, 250, 1),
(6, 'Twin', 750, 350, 2),
(7, 'Suite', 1000, 500, 3),
(8, 'Deluxe', 850, 400, 2),
(9, 'Dorm', 400, 200, 6),
(10, 'Studio', 650, 300, 2);
SELECT * FROM room_types;
```

Result Grid

type_id	type_name	rent	deposit	capacity
1	Single	550	300	1
2	Double	700	350	2
3	Premium	900	450	2
4	Family	1200	600	4
5	Economy	450	250	1
6	Twin	750	350	2
7	Suite	1000	500	3
8	Deluxe	850	400	2
9	Dorm	400	200	6
10	Studio	650	300	2

```
INSERT INTO rooms (room_id, room_no, floor_no, is_occupied, type_id) VALUES
(1, 'A101', 1, TRUE, 2),
(2, 'A102', 1, FALSE, 2),
(3, 'A103', 1, TRUE, 2),
(4, 'A201', 2, FALSE, 1),
(5, 'A202', 2, TRUE, 1),
(6, 'A301', 1, TRUE, 5),
(7, 'B101', 1, FALSE, 4),
(8, 'B201', 2, TRUE, 8),
(9, 'C101', 1, FALSE, 9),
(10, 'C201', 2, TRUE, 10);
SELECT * FROM rooms;
```

Result Grid

room_id	room_no	floor_no	is_occupied	type_id
1	A101	1	1	2
2	A102	1	0	2
3	A103	1	1	2
4	A201	2	0	1
5	A202	2	1	1
6	A301	1	1	5
7	B101	1	0	4
8	B201	2	1	8
9	C101	1	0	9
10	C201	2	1	10

Assignment 1 _ Lam Yoke Yu _...

Limit to 2000 rows

```
102 •  INSERT INTO students (student_id, fname, lname, status, checkin_date, room_id, email) VALUES
103    (1, 'Aiman', 'Rahman', 'Active', '2025-01-10', 1, 'aiman@graduate.utm.my'),
104    (2, 'Siti', 'Zahra', 'Active', '2025-01-12', 3, 'siti@graduate.utm.my'),
105    (3, 'Daniel', 'Tan', 'Active', '2025-02-01', 5, 'daniel@graduate.utm.my'),
106    (4, 'Nurul', 'Izzah', 'Inactive', NULL, NULL, 'nurul@graduate.utm.my'),
107    (5, 'Arif', 'Hakim', 'Active', '2025-01-15', 6, 'arif@graduate.utm.my'),
108    (6, 'Li', 'Wei', 'Active', '2025-02-05', 8, 'li@graduate.utm.my'),
109    (7, 'Amira', 'Sofea', 'Active', '2025-03-01', 10, 'amira@graduate.utm.my'),
110    (8, 'Ravi', 'Kumar', 'Inactive', NULL, NULL, 'ravi@graduate.utm.my'),
111    (9, 'Farah', 'Nadia', 'Active', '2025-02-20', 7, 'farah@graduate.utm.my'),
112    (10, 'Hafiz', 'Azlan', 'Active', '2025-03-10', 9, 'hafiz@graduate.utm.my');
113 •  SELECT * FROM students;
114
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

student_id	fname	lname	status	checkin_date	room_id	email
1	Aiman	Rahman	Active	2025-01-10	1	aiman@graduate.utm.my
2	Siti	Zahra	Active	2025-01-12	3	siti@graduate.utm.my
3	Daniel	Tan	Active	2025-02-01	5	daniel@graduate.utm.my
4	Nurul	Izzah	Inactive	NULL	NULL	nurul@graduate.utm.my
5	Arif	Hakim	Active	2025-01-15	6	arif@graduate.utm.my
6	Li	Wei	Active	2025-02-05	8	li@graduate.utm.my
7	Amira	Sofea	Active	2025-03-01	10	amira@graduate.utm.my
8	Ravi	Kumar	Inactive	NULL	NULL	ravi@graduate.utm.my
9	Farah	Nadia	Active	2025-02-20	7	farah@graduate.utm.my
10	Hafiz	Azlan	Active	2025-03-10	9	hafiz@graduate.utm.my

Result Grid | Form Editor | Field Types

students 6 | Apply | Revert

Assignment 1 _ Lam Yoke Yu _...

Limit to 2000 rows

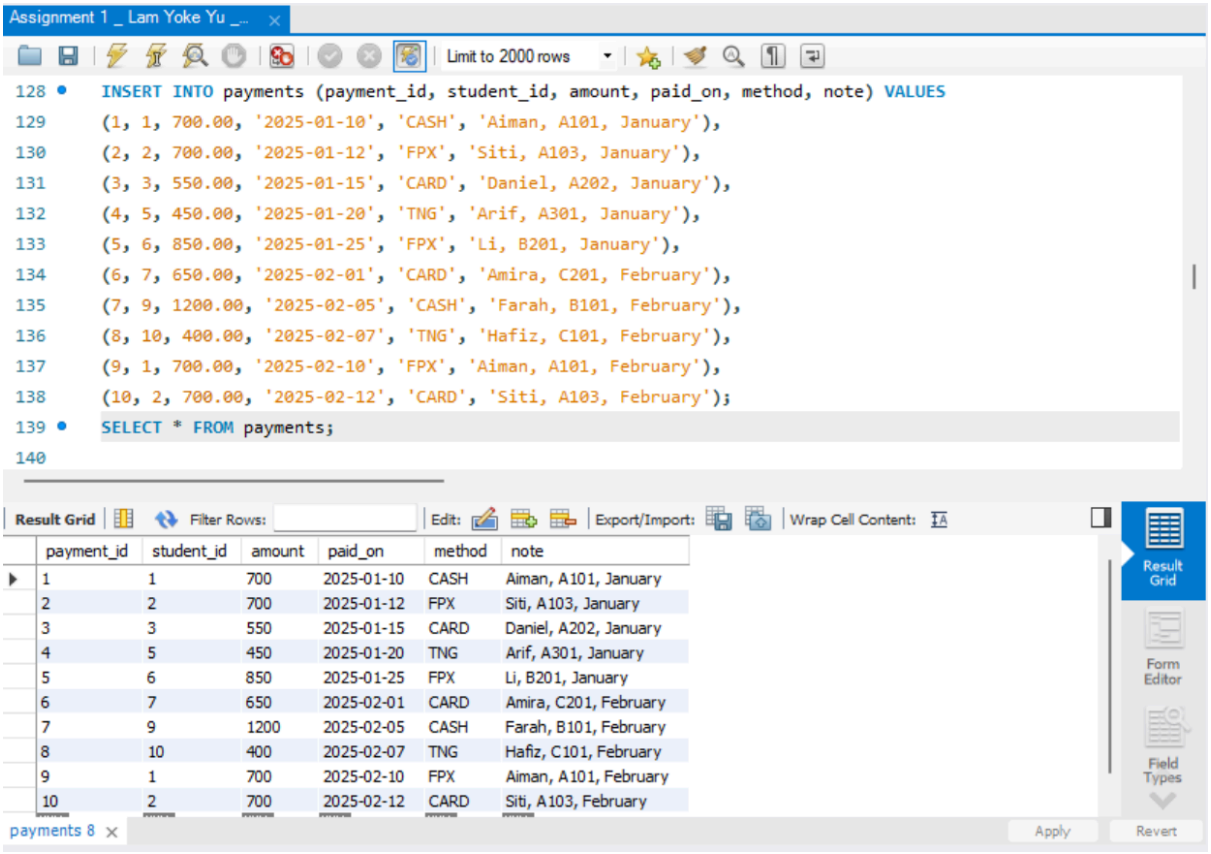
```
115 •  INSERT INTO maintenance (maint_id, room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
116    (1, 7, 'Water leakage', 'HIGH', 'RESOLVED', '2025-01-12', '2025-01-15'),
117    (2, 3, 'Broken chair', 'LOW', 'RESOLVED', '2025-02-03', '2025-02-05'),
118    (3, 6, 'Fan have noise', 'MEDIUM', 'RESOLVED', '2025-03-22', '2025-03-24'),
119    (4, 2, 'Door lock broken', 'HIGH', 'RESOLVED', '2025-03-25', '2025-03-26'),
120    (5, 9, 'Missing pillow', 'LOW', 'OPEN', '2025-04-04', NULL),
121    (6, 4, 'Broken window', 'HIGH', 'RESOLVED', '2025-04-15', '2025-04-17'),
122    (7, 1, 'Unstable WiFi connection', 'LOW', 'OPEN', '2025-05-02', NULL),
123    (8, 10, 'Power socket not functioning', 'MEDIUM', 'RESOLVED', '2025-07-12', '2025-07-13'),
124    (9, 5, 'Window lock rusty', 'LOW', 'OPEN', '2025-08-05', NULL),
125    (10, 8, 'Air conditioner not working', 'MEDIUM', 'OPEN', '2025-11-08', NULL);
126 •  SELECT * FROM maintenance;
127
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

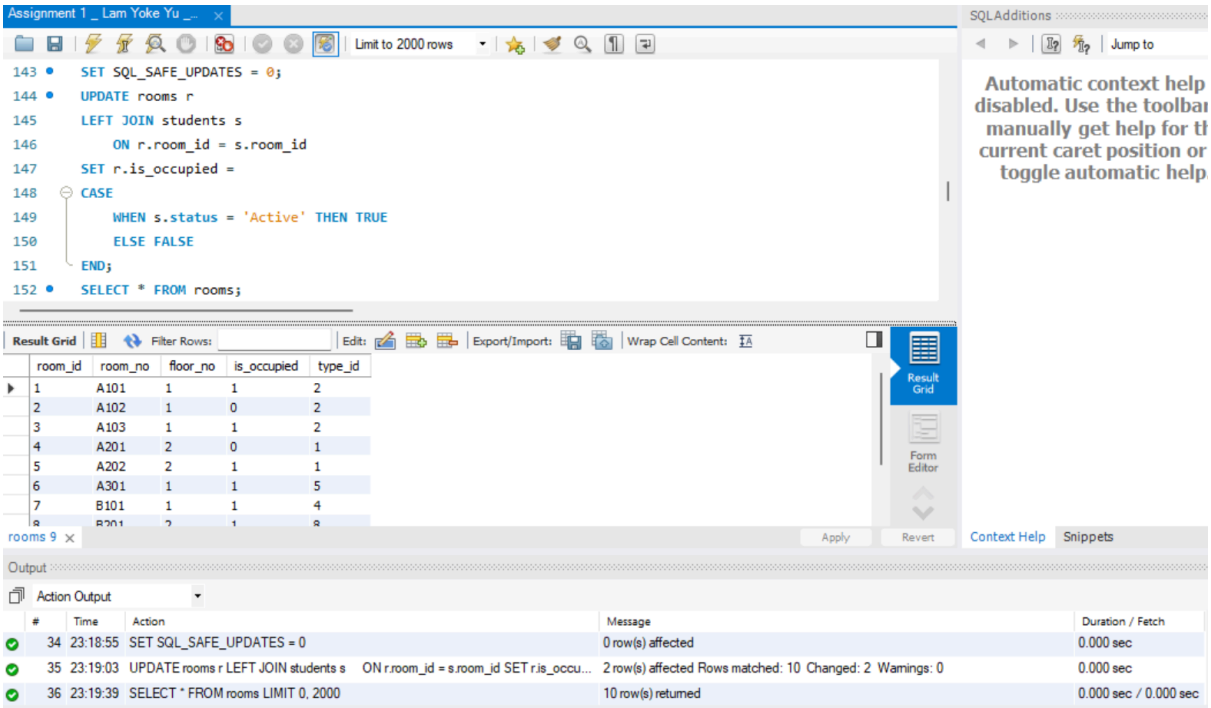
maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	7	Water leakage	HIGH	RESOLVED	2025-01-12	2025-01-15
2	3	Broken chair	LOW	RESOLVED	2025-02-03	2025-02-05
3	6	Fan have noise	MEDIUM	RESOLVED	2025-03-22	2025-03-24
4	2	Door lock broken	HIGH	RESOLVED	2025-03-25	2025-03-26
5	9	Missing pillow	LOW	OPEN	2025-04-04	NULL
6	4	Broken window	HIGH	RESOLVED	2025-04-15	2025-04-17
7	1	Unstable WiFi connection	LOW	OPEN	2025-05-02	NULL
8	10	Power socket not functioning	MEDIUM	RESOLVED	2025-07-12	2025-07-13
9	5	Window lock rusty	LOW	OPEN	2025-08-05	NULL
10	8	Air conditioner not working	MEDIUM	OPEN	2025-11-08	NULL

Result Grid | Form Editor | Field Types

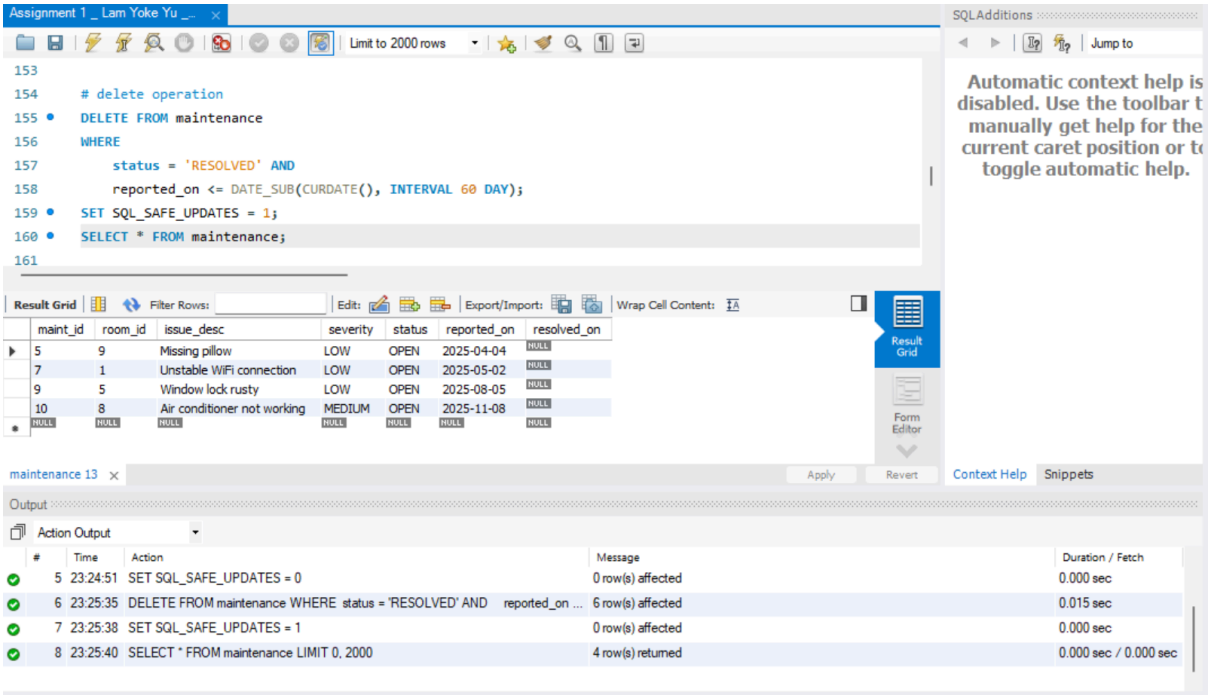
maintenance 7 | Apply | Revert



2. UPDATE and DELETE Operations:

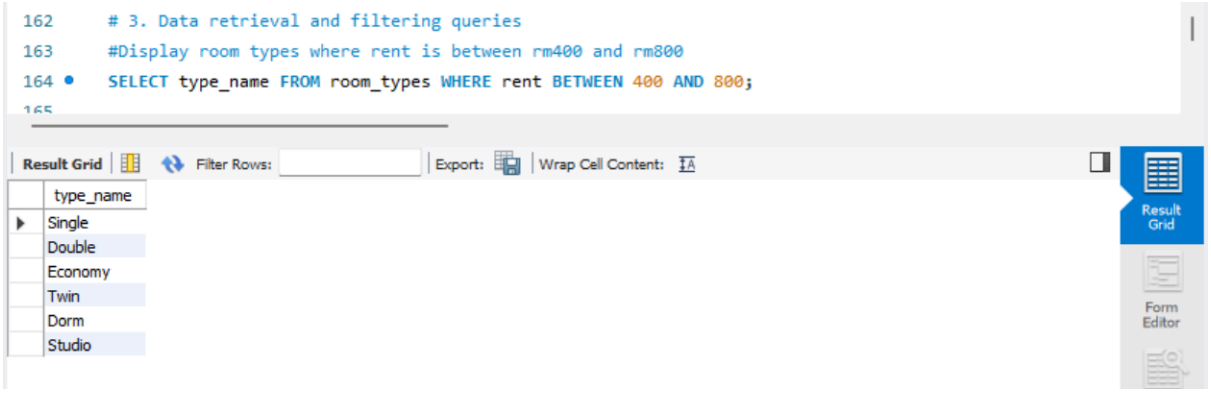


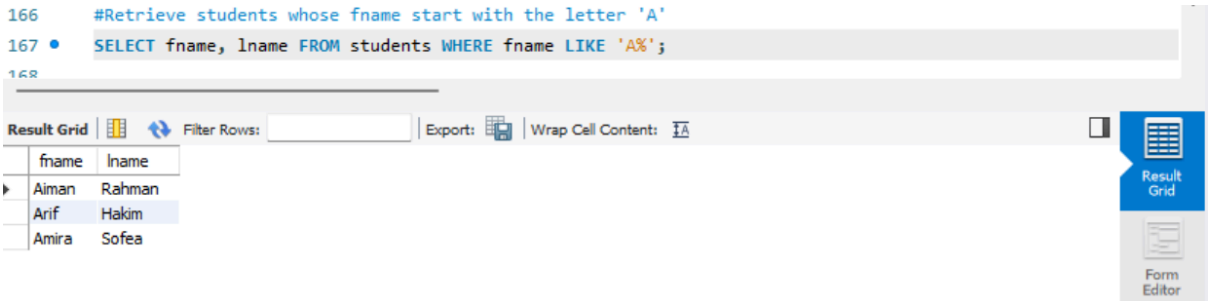
First, safe mode is disabled. Then, `LEFT JOIN` the rooms table with the students table using `room_id`. `is_occupied` in rooms table is updated with `CASE`, which is `SET` to true when student status is active and vice versa.



Using `DELETE FROM`, the row that has status marked as `RESOLVED` and is 60 days old will be deleted from the `maintenance` table. Then, safe mode is enabled for safety.

3. Data Retrieval and Filtering Queries:





```
169    #Display payments where the payment method is FPX or CARD
170    SELECT * FROM payments WHERE method IN ('FPX', 'CARD');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

payment_id	student_id	amount	paid_on	method	note
2	2	700	2025-01-12	FPX	Siti, A103, January
3	3	550	2025-01-15	CARD	Daniel, A202, January
5	6	850	2025-01-25	FPX	Li, B201, January
6	7	650	2025-02-01	CARD	Amira, C201, February
9	1	700	2025-02-10	FPX	Aiman, A101, February
10	2	700	2025-02-12	CARD	Siti, A103, February
NULL	NULL	NULL	NULL	NULL	NULL

Result Grid | Form Editor

```
171
172    #combine AND, OR and NOT
173    SELECT m.room_id, m.issue_desc, m.severity, m.status, r.room_no, r.floor_no
174    FROM maintenance m
175    LEFT JOIN rooms r ON m.room_id = r.room_id
176    WHERE m.status = 'OPEN'
177    AND m.severity != 'LOW'
178        AND (
179            m.severity = 'HIGH'
180            OR  reported_on <= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
181        );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

room_id	issue_desc	severity	status	room_no	floor_no

Result Grid | Form Editor

Result 17 | Read Only

`room_id`, `issue_desc`, `severity`, `status`, `room_no` and `floor_no` are joined from maintenance and rooms using `LEFT JOIN`, where status is open `AND` severity is `NOT` low `AND` severity is high `OR` is left unattained for 30 days or above.

4. Functions and Expressions:

```
184    # Aggregate Function
185    SELECT AVG(rent) AS average_rent
186    FROM room_types;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

average_rent
745

Result Grid | Form Editor

```
188     # String Functions
189 •   SELECT UPPER(CONCAT(fname, ' ', lname)) AS full_name_uppercase, status
190     FROM students;
191
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name_uppercase	status
AIMAN RAHMAN	Active
SITI ZAHRA	Active
DANIEL TAN	Active
NURUL IZZAH	Inactive
ARIF HAKIM	Active
LI WEI	Active
AMIRA SOFEA	Active
RAVI KUMAR	Inactive
FARAH NADIA	Active
HAFIZ AZLAN	Active

Result Grid | Form Editor | Field Types

Result 19 | Read Only

Task 3: Reporting and Aggregation

1. Create `v_room_status`

```
191     # Task 3
192     # 1. Create v_room_status
193 •   CREATE VIEW v_room_status AS
194         SELECT r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity,
195             COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) AS n_occupants,
196             COUNT(CASE WHEN m.status = 'OPEN' THEN 1 END) AS pending_issues,
197             CASE
198                 WHEN COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) = 0 THEN TRUE
199                 ELSE FALSE
200             END AS is_vacant
201         FROM rooms r
202         LEFT OUTER JOIN room_types rt ON (r.type_id = rt.type_id)
203         LEFT OUTER JOIN students s ON (r.room_id = s.room_id)
204         LEFT OUTER JOIN maintenance m ON (r.room_id = m.room_id)
205         GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;
206
207 •   SELECT * FROM v_room_status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Double	700	1	2	1	1	0
A102	Double	700	1	2	0	0	1
A103	Double	700	1	2	1	0	0
A201	Single	550	2	1	0	0	1
A202	Single	550	2	1	1	1	0
A301	Economy	450	1	1	1	0	0
B101	Family	1200	1	4	1	0	0
B201	Deluxe	850	2	2	1	1	0
C101	Dorm	400	1	6	1	1	0
C201	Studio	650	2	2	1	0	0

2. Summary queries

2.1 Total number of students per room type

```
235     # 2.4 Count of OPEN maintenance issues per floor
236 •   SELECT r.floor_no, COUNT(m.maint_id) AS open
237     FROM maintenance m
238     LEFT OUTER JOIN rooms r ON (m.room_id = r.room_id)
239     WHERE m.status = 'OPEN'
240     GROUP BY r.floor_no
241     HAVING COUNT(m.maint_id) > 2;
242
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

floor_no	open

After adding several entries the results are as follow:

```sql
# 2. Summary Queries
# 2.1 Total number of students per room type
SELECT rt.type_name, COUNT(CASE WHEN s.status = 'ACTIVE' THEN 1 END) AS n_occupants
FROM room_types rt
LEFT OUTER JOIN rooms r ON (r.type_id = rt.type_id)
LEFT OUTER JOIN students s ON (r.room_id = s.room_id)
GROUP BY rt.type_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

type_name	n_occupants
Single	1
Double	2
Premium	0
Family	1
Economy	1
Twin	0
Suite	0
Deluxe	1
Dorm	1
Studio	1

2.2 Average rent and total deposit per room type

This statement also used the `UPPER()` and `CASE` as required in No. 3

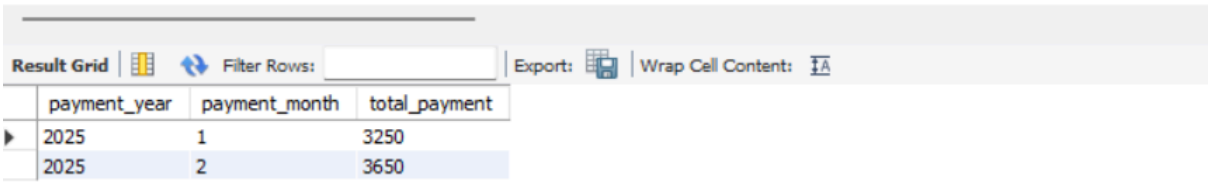
```sql
# 2.2 Average rent and total deposit per room type
SELECT UPPER(rt.type_name),
    ROUND(AVG(rt.rent), 2) AS avg_rent,
    ROUND(SUM(rt.deposit), 2) AS total_deposit,
    CASE
        WHEN AVG(rt.rent) > 1000 THEN 'High'
        WHEN AVG(rt.rent) > 500 THEN 'Medium'
        ELSE 'Low'
    END AS rent
FROM students s
LEFT OUTER JOIN  rooms r ON (r.room_id = s.room_id)
LEFT OUTER JOIN room_types rt ON (r.type_id = rt.type_id)
WHERE s.status = 'Active'
GROUP BY rt.type_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

UPPER(rt.type_name)	avg_rent	total_deposit	rent
DOUBLE	700	700	Medium
SINGLE	550	300	Medium
ECONOMY	450	250	Low
DELUXE	850	400	Medium
STUDIO	650	300	Medium
FAMILY	1200	600	High
DORM	400	200	Low

2.3 Monthly payment totals grouped by year & month

```
227  # 2.3 Monthly payment totals grouped by year & month
228  SELECT
229      YEAR(paid_on) AS payment_year,
230      MONTH(paid_on) AS payment_month,
231      ROUND(SUM(amount), 2) AS total_payment
232  FROM payments
233  GROUP BY YEAR(paid_on), MONTH(paid_on);
234
```



payment_year	payment_month	total_payment
2025	1	3250
2025	2	3650

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (`PK, FK, NOT NULL, UNIQUE`). Logical order and referential integrity preserved.	3	
	`ALTER TABLE` (add email `UNIQUE`) and `DROP TABLE` demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data `UPDATE/DELETE` executed correctly with logical results.	1	
	Accurate use of `WHERE, BETWEEN, LIKE, IN, AND/OR/NOT` queries.	3	
	Correct use of aggregate and string functions (`COUNT, CONCAT, and more).`	2	
Task 3 Reporting & Aggregation	`VIEW` creation	2	
	Use of `GROUP BY, HAVING, ROUND, and CASE` with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
		Total (20)	